Intrusion Detection System — Preprocessing & CNN-BiLSTM Training Methodology

> **Scope.** This document covers the data pipeline and model training procedure for the network intrusion detection component of the IDS-Blockchain project. It is written to document every design decision with its technical justification so the approach can be reproduced and evaluated independently.

---

1. Dataset

Source: CSE-CIC-IDS2018 (Canadian Institute for Cybersecurity, 2018).
Format: Ten daily CSV files (`*TrafficForML_CICFlowMeter.csv`), approximately 16 GB total on disk.
Feature extraction: Performed externally by CICFlowMeter, which reduces raw packet captures to ~80 per-flow statistical features (byte counts, packet lengths, inter-arrival times, flag counts, etc.) per network flow.
Classes: 15 traffic categories — one benign class and 14 attack families spanning DoS, DDoS, brute-force, web attacks, botnet, and infiltration.

---

2. Preprocessing Pipeline

Script: `ai_detection/src/data_processing/preprocess.py`

The pipeline processes the full 16 GB dataset in streaming chunks to stay within RAM limits, applies cleaning, reduces features, scales, and writes split tensors ready for the model.

2.1 Column Selection

Three categories of columns are removed before any numeric processing:

Category	Columns removed	Justification
Metadata / leakage	`Timestamp, Flow ID, Src IP, Dst IP, Src Port, Dst Port, Protocol`	These are identifiers, not traffic statistics. Including IP/port tuples would cause the model to memorize specific hosts rather than learn attack patterns — a form of data leakage.
Zero-variance (literature-confirmed)	10 columns including `Bwd PSH Flags`, `Fwd URG Flags`, `CWE Flag Count`, `Fwd/Bwd Byts/Pkts/Blk Avg`	Gocs & Johanyak (2024) confirmed these features carry the same constant value across the entire dataset. A zero-variance feature contributes zero mutual information with any target and only adds noise after scaling.
Label column	`Label`	Kept separately; not part of the feature matrix.

Only columns present in **all ten CSVs** are retained (intersection of column sets across files), ensuring consistency across recording days.

2.2 Label Normalization

Each CSV uses inconsistent label strings (e.g., `"DDOS attack-HOIC"`, `"DDoS attack-HOIC"`, `"BENIGN"`, `"Benign"`). A deterministic mapping table unifies all variants to 15 canonical names before any other processing. Rows with unmapped labels are dropped.

```
"DoS attacks-GoldenEye"  →  "DoS-GoldenEye"
"SSH-Bruteforce"         →  "SSH-BruteForce"
"Infilteration"          →  "Infiltration"     (typo in source data)
"Bot"                    →  "Botnet"
... (full map in preprocess.py LABEL_MAP)
```

2.3 Chunk-Based Streaming Load and Class Capping

The dataset is too large to hold in RAM simultaneously (~16 GB). Each CSV is read in chunks of 200,000 rows. For each chunk, after cleaning, rows are accumulated per class in per-class buffers, each capped at a fixed maximum:

Class	Cap
Benign	400,000 flows
All attack classes	200,000 flows each

Why a higher Benign cap? In live network traffic, benign flows represent the vast majority. Cutting Benign too aggressively relative to attack classes distorts the training prior, and the model learns a marginal class distribution that does not reflect deployment. The 400,000 cap keeps Benign as the plurality class while preventing it from drowning the minority attacks — matching realistic traffic proportions more closely than hard balancing.

Why cap attacks at all? Without capping, larger attack families (LOIC-HTTP, Hulk) would dominate training, and the model would spend most gradient steps on these, underserving small classes like SQL-Injection or Infiltration.

Classes with fewer than 50 total flows after capping are dropped as statistically insufficient for a held-out test evaluation.

2.4 Per-Chunk Cleaning

For each chunk the following steps are applied in order:

1. **Strip whitespace** from column names (common formatting inconsistency in CICFlowMeter output).
2. **Coerce all feature columns to numeric** (`pd.to_numeric(errors='coerce')`). Non-parseable strings become `NaN`.
3. **Drop rows with `Inf` or `NaN`** values. These arise from division-by-zero inside CICFlowMeter when packet counts are zero.
4. **Drop rows with negative `Flow Duration`** — physically impossible, indicating a timestamp recording error.
5. **Drop exact duplicate rows** — repeated identical flow tuples provide no additional information and inflate certain class counts.
6. **Cast surviving feature columns to `float32`** to halve memory footprint vs `float64`.

2.5 Post-Concat Feature Reduction

After all chunks are combined into a single DataFrame:

Remaining zero-variance check: a second standard-deviation pass is run on the combined dataset. Any feature with `std == 0` across the actual retained rows is dropped (this catches

features that became constant after the per-class capping).

Correlation-based removal: the absolute Pearson correlation matrix is computed, the upper triangle is inspected, and any column with `|r| > 0.98` against any previously retained column is removed. Rationale: near-perfectly correlated features are linear combinations of each other — they add parameter count, memory, and training variance without adding information. Threshold 0.98 (not 1.0) is used because CICFlowMeter sometimes produces features that are proportional rather than identical (e.g., byte count vs. bit count).

2.6 Outlier Clipping (Winsorization)

Network traffic features have extreme outliers. A flow with a `Flow Bytes/s` of 10^{12} caused by a zero-duration edge case or sensor artifact would shift the scaler's mean and standard deviation dramatically, compressing the representation of all normal flows into a tiny numeric range.

Clipping is applied by replacing values below the **1st percentile** and above the **99th percentile** with those boundary values — computed per feature on the full combined dataset before the split. This is a form of Winsorization. The percentile bounds are not stored as a separate artifact; the clipped values are simply in the `X_*.npy` arrays. The decision to clip at 1st/99th rather than, say, 3σ was made because the feature distributions are heavily right-skewed (most network features are non-negative with a long tail), and σ-based clipping would still retain pathological values.

2.7 Stratified Train / Validation / Test Split

The combined, shuffled DataFrame is split **stratified by class label** in a 70 / 15 / 15 ratio:

```
X_train, X_tmp  = train_test_split(X, y, test_size=0.30, stratify=y, random_state=42)
X_val,   X_test = train_test_split(X_tmp, y_tmp, test_size=0.50, stratify=y_tmp, rand
```

Stratification is mandatory: without it, small classes (< 200 samples after capping) could be absent from the test set entirely, making per-class F1 scores undefined.

The same `random_state=42` is used throughout for reproducibility.

2.8 Z-Score Standardization

A `sklearn.preprocessing.StandardScaler` is fit **exclusively on** `X_train`, then applied to `X_val` and `X_test`. This is a critical discipline: fitting on all data would leak distributional information from the validation and test sets into the scaler, inflating apparent generalization performance.

```
x'_train = (x_train − μ_train) / σ_train
x'_val   = (x_val   − μ_train) / σ_train
x'_test  = (x_test  − μ_train) / σ_train
```

After scaling, any residual `NaN` or `±Inf` (possible in constant-variance features not caught by earlier checks) is zeroed out with `np.nan_to_num`.

StandardScaler (z-score) is preferred over MinMaxScaler here because the model is a deep network with gradient-based optimization. Z-score normalization centers activations near zero and maintains unit variance, which stabilizes gradients and speeds convergence. MinMaxScaler is sensitive to a single outlier setting the min/max, and despite Winsorization there remain heavy-tailed features where z-score is more robust.

2.9 Class Weights

Inverse-frequency class weights are computed via `sklearn.utils.class_weight.compute_class_weight('balanced', ...)` on `y_train`. These are saved to `data/processed/class_weights.npy` and used to parameterize the focal loss alpha (see §5.1).

2.10 Saved Artifacts

File	Contents
`data/processed/X_train.npy`	(N_train, F) float32 feature matrix
`data/processed/X_val.npy`	(N_val, F) float32
`data/processed/X_test.npy`	(N_test, F) float32
`data/processed/y_*.npy`	(N,) int64 class indices
`data/processed/class_weights.npy`	(C,) float32 inverse-frequency weights
`data/processed/scaler.pkl`	Fitted `StandardScaler` — must be applied to any new flow at inference
`data/processed/label_encoder.pkl`	`LabelEncoder` mapping string names ↔ integer indices
`data/processed/class_names.json`	Ordered list of 15 class names

File	Contents
`data/processed/feature_names.json`	Ordered list of retained feature names

---

3. Data Representation for the Sequence Model

Core invariant: one network flow = one sample. No sliding windows.

CICFlowMeter already aggregates an entire TCP/UDP conversation into a single feature vector of ~80 statistics. A network flow is not a time series of observations — it is a single statistical summary. Applying a sliding window over rows would mean constructing artificial "sequences" from successive, independently shuffled flows, which are not temporally related. Such a construction would be physically meaningless and would cause label leakage (a benign flow's statistics "predicting" the next malicious flow is coincidence, not causality).

3.1 Pseudo-Sequence Reshape

The LSTM architecture requires a 3D input tensor of shape `(Batch, Timesteps, Features_per_step)`. Since the input is a single 1D feature vector per flow, it is reshaped into a pseudo-sequence by grouping features into chunks of size `C = 4`:

```
F features → padded to pad_to = ceil(F/4) × 4
→ reshaped to (T = pad_to/4, C = 4)
```

For example, if F = 46 features: `pad_to = 48`, `T = 12`, `C = 4`. The resulting shape is `(12, 4)`.

This gives the CNN and LSTM a structured sequence to iterate over — not a real time series, but a structured factorization of the feature space. Each "timestep" corresponds to a group of four correlated flow statistics. The CNN front-end (§4.1) can learn local interactions within and between adjacent chunks before the LSTM integrates across all chunks.

---

4. Model Architecture: CNN-BiLSTM v2

Script: `ai_detection/src/training/train_v2.py`
Checkpoint: `models/bilstm_ids_v2.pt`

The full forward pass:

```
Input (B, T=12, 4)
  → CNN front-end
  → BiLSTM
  → Multi-head attention pooling
  → Classification head
  → Logits (B, 13)
```

4.1 CNN Front-End

```
nn.Conv1d(in_channels=4,   out_channels=64,  kernel_size=3, padding=1)  # GELU
nn.Conv1d(in_channels=64,  out_channels=128, kernel_size=3, padding=1)  # GELU
```

The input sequence `(B, T, C)` is transposed to `(B, C, T)` before convolution (Conv1d operates over the T dimension). Two stacked 1D convolutions with kernel size 3 allow the model to learn local interactions across adjacent feature groups before passing the enriched representation to the LSTM.

Why a CNN front-end? The published literature on CIC-IDS-2018 consistently reports that CNN-LSTM hybrids outperform pure BiLSTM models. The CNN acts as a learned feature extractor that captures short-range correlations in the pseudo-sequence before the LSTM captures long-range dependencies across all timesteps. Training from scratch, the CNN front-end also provides a richer gradient signal early in training because convolutions have smaller receptive fields and saturate less than LSTM cells at initialization.

Why kernel size 3 and not 5 or 7? With T=12, a kernel of size 3 covers adjacent triplets of feature groups without crossing more than a quarter of the sequence. Larger kernels would overlap excessively at this sequence length, effectively turning the conv into a global pooling operation and eliminating its advantage.

No BatchNorm or pooling between conv layers. The sequence is already short (T=12); spatial pooling would halve it to T=6 before the LSTM even starts, losing the structured progression of feature groups.

4.2 BiLSTM

```
nn.LSTM(input_size=128, hidden_size=128, num_layers=2,
        batch_first=True, bidirectional=True,
        dropout=0.30)  # between layers
```

After the CNN, the representation is `(B, T, 128)`. The BiLSTM produces `(B, T, 256)` (128 forward + 128 backward hidden states concatenated at each timestep).

Bidirectionality. Because this is a per-flow classification task and the entire pseudo-sequence is always present at inference time (not streaming), the model has access to the complete sequence in both directions. A bidirectional LSTM can integrate context from later feature groups when processing earlier ones, which is not possible with a unidirectional LSTM. For real-time packet-by-packet inference a unidirectional LSTM would be required, but for post-flow classification there is no causality constraint.

2 layers, hidden=128. A deeper or wider LSTM (e.g., 3 layers, hidden=192, as in earlier experiments) is over-parameterized for sequences of length T=12. With 12 timesteps, the gradient signal must propagate through at most 12 steps regardless of depth. More layers add parameters without providing longer effective memory, and on a short sequence they introduce additional vanishing gradient risk and regularization burden. hidden=128 gives the BiLSTM a 256-dimensional output that is expressive without being redundant.

4.3 Multi-Head Attention Pooling

```
class MultiHeadAttentionPool(nn.Module):
    # Learnable query vector → cross-attends to LSTM output sequence
    # Output: (B, 256) — single vector per flow
```

A single learnable query vector of shape `(1, 1, 256)` is expanded over the batch and used as the query in a `nn.MultiheadAttention(embed_dim=256, num_heads=4)` call against the LSTM output sequence as both key and value. The output is a single `(B, 256)` vector per flow, followed by LayerNorm.

Why attention pooling instead of last-hidden-state or mean pooling?

- *Last hidden state:* discards all but the final timestep's representation, ignoring what earlier timesteps may have contributed.
- *Mean pooling:* treats all timesteps equally, which is appropriate only if every feature group is equally informative for classification.
- *Attention pooling:* learns a weighted combination of timesteps, allowing the model to focus on the feature groups that are most discriminative for a given flow. With 4 heads, different heads can specialize in different aspects of the traffic statistics (e.g., one head attending to timing features, another to byte counts).

LayerNorm after attention stabilizes the pooled representation before the classification head.

4.4 Classification Head

```
nn.Linear(256, 128) → nn.GELU() → nn.Dropout(0.30) → nn.Linear(128, num_classes)
```

A two-layer MLP reduces the 256-dimensional pooled representation to class logits. GELU activation is used over ReLU because it provides a smooth gradient at zero (no hard threshold), which benefits the attention-pooled representation that may have near-zero activations.

Dropout (p=0.30) is applied after the first linear transformation, providing regularization specific to the classification head without interfering with the recurrent weights.

---

5. Class Imbalance Strategy

The 15 attack classes in CIC-IDS-2018 span several orders of magnitude in sample count. After capping, Benign has ~400,000 flows while SQL-Injection or BruteForce-Web may have fewer than 100. Naive training on this distribution produces models that predict Benign for nearly everything.

5.1 The Rebalancing Constraint: At Most Two Levers

A critical failure mode was identified during development: **stacking three simultaneous rebalancing mechanisms** (SMOTE synthetic oversampling + WeightedRandomSampler + class-weighted focal loss alpha) gives the model an effectively uniform training prior. When the training prior is uniform but the test distribution is highly skewed, the model cannot calibrate its output probabilities to the true class frequencies. The observable symptom is Benign recall collapsing to ~0.29 (the model refusing to predict Benign because it was never trained to expect that Benign is overwhelmingly likely).

The rule derived from this failure: **use at most two rebalancing levers simultaneously**, and treat logit-adjusted training (§5.3) as the primary lever.

Current Option B defaults (in `train_v2.py`):

```
USE_SAMPLER  = False       # WeightedRandomSampler disabled
FOCAL_GAMMA  = 2.0
# focal alpha = sqrt(class_weights) / mean(sqrt(class_weights))
# No SMOTE — LA is the sole rebalancing lever
```

5.2 Web Attack Class Merging (15 → 13 Classes)

Three attack classes — `BruteForce-Web`, `BruteForce-XSS`, and `SQL-Injection` — have extremely few test examples (12, 34, and 86 respectively after the stratified split). At these sample counts, direct 15-class softmax classification is statistically unreliable: a class with 12 test flows produces F1 scores with high variance and near-zero expected value regardless of model quality.

Solution: during training, these three classes are merged into a single `WebAttack` class (13-class training). The loss gradients for web attacks are pooled across all three sub-types, giving the model a meaningful signal for "web attack" detection.

At inference, a separate `LogisticRegression` classifier (`web_splitter_v2.pkl`) is trained exclusively on the training flows belonging to the three web attack sub-types. When the CNN-BiLSTM predicts `WebAttack`, the splitter is called on the raw feature vector to determine which of the three sub-types to report. This two-stage inference preserves the 15-class output for reporting purposes.

```
CNN-BiLSTM (13 classes)
  ├─ if predicted class ≠ WebAttack: use the prediction directly
  └─ if predicted class == WebAttack: call web_splitter → {BruteForce-Web, BruteForce
```

5.3 Logit-Adjusted Focal Loss

Reference: Menon et al., "Long-tail learning via logit adjustment," NeurIPS 2021.

The standard focal loss for multi-class classification is:

$$\mathcal{L}_{FL}(p_t) = -(1 - p_t)^{\gamma} \log(p_t)$$

where p_t is the predicted probability for the true class and $\gamma = 2.0$ is the focusing parameter (down-weighting easy examples).

The logit adjustment modifies the loss computation as follows. Let π_c be the training-set frequency of class c (computed from the 13-class remapped `y_train`). At each training step, before computing the focal loss, the per-class logit adjustment is applied:

$$\tilde{z}_c = z_c + \tau \cdot \log(\pi_c), \quad \tau = 1.0$$

The focal loss is then computed on the adjusted logits $\tilde{z}$. The effect is that the model must produce raw logits for rare classes that are approximately $\tau \cdot |\log(\pi_c)|$ higher than for common classes, just to compete equally after adjustment. For a class like `SQL-Injection` with $\pi_c \approx 10^{-3}$, this is a forced upward push of ~6.9 logit units — the model cannot simply collapse the rare class to zero.

Why LA and not SMOTE? SMOTE generates synthetic minority-class samples by interpolating between existing samples in feature space. For network flows, this means creating artificial `SQL-Injection` feature vectors as linear combinations of real ones. Because the SQL-Injection feature space overlaps significantly with Benign (many SQL injection flows look like normal HTTP traffic to a flow-level classifier), the synthetic samples bleed into the Benign region and corrupt the decision boundary. LA avoids synthetic data entirely: it corrects the class imbalance at the logit level without manufacturing data points.

Natural-prior inference correction: because the model is trained with a logit shift of $\tau \cdot \log(\pi_c)$, the raw logits at inference have learned to over-predict rare classes relative to the true posterior. The same shift must be re-applied at inference:

$$\hat{y} = \arg\max_c \left[z_c + \tau \cdot \log(\pi_c) \right]$$

This is called the **natural-prior logit adjustment** at inference. Omitting it — using raw argmax — produces a 15-class macro-F1 of approximately 0.64. Applying it recovers the calibrated macro-F1 of **0.7711**. The log-prior vector is derived from the training set class frequencies and stored alongside the checkpoint.

---

6. Training Protocol

Script: `ai_detection/src/training/train_v2.py`

6.1 Optimizer and Learning Rate Schedule

```
optimizer = torch.optim.AdamW(model.parameters(), lr=1.5e-3, weight_decay=2e-5)
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer, max_lr=1.5e-3, epochs=30,
    steps_per_epoch=len(train_loader), pct_start=0.1,
)
```

AdamW (Adam with decoupled weight decay) is used instead of plain Adam. Standard Adam applies L2 regularization to the adaptive gradient estimates, which distorts the effective regularization strength. AdamW applies weight decay directly to the parameters after the gradient step, providing cleaner regularization behavior.

OneCycleLR linearly warms the learning rate from near-zero to `max_lr` over the first 10% of training steps (`pct_start=0.1`), then decays it with a cosine annealing schedule. The warm-up phase prevents large early gradient steps from pushing the randomly initialized model weights into a bad local optimum during the first epoch. The cosine decay allows fine-grained convergence in the later epochs.

6.2 Automatic Mixed Precision (AMP)

```
scaler = torch.amp.GradScaler("cuda")

with torch.amp.autocast("cuda"):
    logits = model(xb)
    loss = criterion(logits, yb)

scaler.scale(loss).backward()
scaler.unscale_(optimizer)
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
scaler.step(optimizer)
scaler.update()
```

AMP computes forward and backward passes in `float16` where numerically safe, and accumulates gradients in `float32`. This halves VRAM usage per batch and approximately doubles throughput on CUDA tensor cores. The `GradScaler` dynamically scales the loss before backprop to prevent float16 underflow, then unscales before gradient clipping. This allows `BATCH_SIZE = 2048` on the RTX 2070 Max-Q (8 GB VRAM), whereas without AMP the safe limit is 1024.

6.3 Gradient Clipping

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

Gradient norm clipping prevents exploding gradients during the early warm-up phase and whenever the loss surface is steep (e.g., after a misclassified batch of rare classes whose logit-

adjustment forced unusually high loss values). The threshold of 1.0 is the standard for LSTM-based models.

6.4 Exponential Moving Average (EMA)

```
EMA_DECAY = 0.999

# After each optimizer step:
θ_shadow ← 0.999 · θ_shadow + 0.001 · θ_live
```

EMA maintains a shadow copy of the model weights, exponentially averaged over training history. At the end of each epoch, the EMA weights are temporarily swapped into the model for validation F1 computation, then restored. The best EMA state (by validation macro-F1) is saved as the final checkpoint.

Why EMA? Stochastic gradient descent causes the model weights to oscillate around a local optimum. The EMA average smooths out these oscillations, effectively averaging over a neighborhood of weight configurations rather than committing to the last training step. In practice, EMA checkpoints generalize better than the last-iterate checkpoint, especially for minority classes where individual batches may push weights in conflicting directions.

6.5 Early Stopping

```
PATIENCE = 8
```

Training halts if the validation macro-F1 does not improve for 8 consecutive epochs. The best checkpoint (lowest patience counter) is always restored, so the final model corresponds to the epoch with the best validation performance rather than the epoch at which training terminated. Maximum training is capped at 30 epochs.

6.6 Batch Size

```
BATCH_SIZE = 2048  # training
BATCH_SIZE * 2     # inference (no gradients, lower VRAM requirement)
```

Larger batches provide more stable gradient estimates for minority classes, which appear rarely within any given batch. At batch size 2048, a class with training frequency 1% has an expected

count of ~20 samples per batch — sufficient for a meaningful gradient contribution. At smaller batch sizes (e.g., 128), a minority class may contribute zero samples to some batches entirely.

---

7. Inference Pipeline

At inference time, a new network flow goes through the following steps:

1. **Feature extraction:** CICFlowMeter produces a vector of the same features used during training (determined by `feature_names.json`).
2. **Standardization:** the saved `scaler.pkl` (fit on training data) transforms the raw features to z-scores.
3. **Reshape:** the feature vector is padded to `pad_to` and reshaped to `(1, T, 4)`.
4. **Forward pass:** the CNN-BiLSTM produces raw logits of shape `(1, 13)`.
5. **Natural-prior correction:** $\tilde{z}_c = z_c + \tau \cdot \log(\pi_{train,c})$ is applied.
6. **Class prediction:** `argmax(tilde_z)` selects the 13-class prediction.
7. **Web attack splitting:** if the prediction is `WebAttack`, the `web_splitter_v2.pkl` logistic regression is applied to the original (standardized) feature vector to resolve the 3-way sub-type.
8. **Output:** one of the 15 class labels is returned.

Step 5 is mandatory. Skipping natural-prior correction degrades macro-F1 by approximately 0.13 absolute points because the model was trained under a distorted prior and its raw logits reflect that distortion.

---

8. Saved Model Artifacts

File	Contents
`models/bilstm_ids_v2.pt`	CNN-BiLSTM weights + architecture metadata + label maps
`models/web_splitter_v2.pkl`	Logistic regression splitter for 3-way web attack classification

The checkpoint stores all architecture hyperparameters inline (`chunk`, `cnn_ch`, `hidden`, `lstm_layers`, `num_classes`, `dropout`, `n_heads`, `pad_to`, `num_features`) so the model can be reconstructed without access to the training script.

---

9. Summary of Design Decisions

Decision	Chosen option	Rejected alternative	Reason
Flow representation	Per-flow, single sample	Sliding window over rows	Sliding windows over shuffled flows create physically meaningless sequences; no temporal relation between successive rows
Feature input shape	Pseudo-sequence `(T=12, C=4)`	Flat vector `(F,)`	LSTM requires 3D input; grouping into chunks gives CNN a structured local context
Scaling	Z-score (StandardScaler, train-only fit)	MinMaxScaler	Z-score more robust to heavy-tailed distributions; MinMaxScaler sensitive to outlier extremes
Rebalancing	Logit-adjusted focal loss only	SMOTE + sampler + focal (triple-stack)	Triple-stack gives uniform training prior → Benign recall collapses; LA avoids synthetic data and corrects at the logit level
Web attacks	Merge to 1 training class + splitter	Direct 15-class training	< 100 test examples per class makes direct F1 meaningless; pooled training gives sufficient gradient
Inference correction	`argmax(z + log π_train)`	Raw argmax	Model trained under LA-shifted prior; raw argmax recovers the shifted distribution; correction required to calibrate
LSTM direction	Bidirectional	Unidirectional	Post-flow classification — full sequence always available; bidirectional can use context from all feature groups
Attention pooling	4-head cross-attention	Mean / last-hidden-state	Learns weighted combination of timesteps; different heads specialize in different feature groups

Decision	Chosen option	Rejected alternative	Reason
EMA	decay=0.999	No EMA	Smooths weight oscillations from stochastic updates; checkpoint generalizes better than last iterate
Optimizer	AdamW	Adam	Decoupled weight decay provides cleaner L2 regularization than Adam's gradient-coupled version
LR schedule	OneCycleLR (warm-up + cosine decay)	Constant LR / StepLR	Warm-up prevents early instability; cosine decay enables fine-grained convergence

---

Document reflects the state of `ai_detection/src/data_processing/preprocess.py` and `ai_detection/src/training/train_v2.py` as of April 2026.